

The OAuth2 Plugin

Authorization Server — OAuth 2.1 + OpenID Connect

authorization_code (+PKCE) • client_credentials • refresh_token • password • device_code

A complete guide to using and implementing `Plugins\OAuth2`

solves: `oauth.server`

Native • dependency-free (on firebase/php-jwt) • access tokens are platform JWTs

Contents

1	What the OAuth2 plugin is	2
1.1	OAuth2 vs Auth — which do I use?	2
2	The client model	2
3	Grants	2
4	Endpoints	3
5	Tokens	3
6	Application services + ports	4
7	Self-service management API	4
7.1	Clients (/oauth/clients)	4
7.2	Authorized apps (/oauth/authorized-tokens)	4
7.3	Scopes (/oauth/scopes)	4
8	Usage recipes	4
8.1	Register a client (CLI or API)	4
8.2	Authorization code + PKCE (public SPA)	5
8.3	Client credentials (server-to-server)	5
8.4	Refresh + rotation	5
8.5	Device code (TV / CLI)	5
8.6	Introspect / revoke (resource server & logout)	5
8.7	Consuming an OAuth token in a resource server	5
9	CLI	6
10	Configuration (env)	6
11	Rules — do and don't	6

1 What the OAuth2 plugin is

Plugins\OAuth2 (solves: `oauth.server`) is a native, dependency-free **OAuth 2.1 + OpenID Connect authorization server** — the piece a first-party Plugins\Auth module cannot provide: **third-party / delegated** access, where a *separate application* is authorized to act on a user's behalf with a limited set of scopes.

It reuses `firebase/php-jwt` (already a kernel dependency) — **no new vendor packages**. Its access tokens are ordinary platform JWTs, so they are verified by Auth's `JwtAuthLayer` with zero extra resource-server code.

```
requires: ["database.management", "crypto.services", "user.management", "view.rendering"]
exposes:  ["Plugins\OAuth2\Application\Ports\ClientStore"]
```

Placement. OAuth2 is a **CENTRAL** / control-plane concern — serve `/oauth/*` on the **apex / central host**, never on tenant sub-domains. In host-tenancy mode set `TENANCY_BASE_DOMAINS` so the apex resolves to central. All control-plane tables (`oauth_clients`, `oauth_auth_codes`, `oauth_refresh_tokens`, `oauth_scopes`, `oauth_device_codes`) are pinned to the central connection.

1.1 OAuth2 vs Auth — which do I use?

Use Plugins\Auth	Use Plugins\OAuth2
Your own app / SPA / mobile logging in	A THIRD-PARTY app acting for a user
User API keys (PATs, <code>/auth/tokens</code>)	Delegated client tokens (<code>/oauth/token</code>)
Sessions, remember-me, refresh sessions	Authorization-code / device / client-credentials grants
<code>personal_access_tokens</code> (user keys)	<code>oauth_clients</code> (APPLICATIONS)

2 The client model

An OAuth *client* is a registered application (table `oauth_clients`):

- **Confidential** — holds a hashed secret; may use any grant; ALWAYS authenticates (HTTP Basic or body secret, compared with `hash_equals`).
- **Public** (SPA / mobile) — no secret; identified by `client_id` + **PKCE**; must use `authorization_code` + PKCE.
- `redirect_uris` are matched **exactly** (OAuth 2.1 — no wildcards / prefixes), validated *before* any error redirect.
- `scopes` (empty = any registered scope), `grant_types`, `owner_id` (the user who registered it — powers self-service mgmt).

3 Grants

Grant	Notes
<code>authorization_code</code> (+PKCE)	exact-match <code>redirect_uri</code> ; PKCE mandatory for public clients (S256/plain); codes random, hashed, 60s, single-use (atomic consume)
<code>client_credentials</code>	confidential clients only; no refresh token; <code>sub</code> = <code>client_id</code>

<code>refresh_token</code>	rotating + family reuse-detection (replay burns the family); scope narrowing only
<code>password</code>	confidential client; verifies via <code>ResourceOwnerVerifier</code> (User module); deprecated by OAuth 2.1
<code>device_code</code>	RFC 8628; <code>authorization_pending</code> / <code>slow_down</code> (interval-enforced) / <code>access_denied</code> / <code>expired_token</code> ; single redemption

4 Endpoints

Method • Path	Purpose
GET/POST /oauth/authorize	auth-code consent (session-auth gated; request stored server-side; the form carries only an opaque <code>authz_id</code>)
POST /oauth/token	token endpoint (all grants)
POST /oauth/device_authorization	device-code start (<code>device_code</code> + <code>user_code</code>)
GET/POST /oauth/device	device user-verification page
GET /oauth/userinfo	OIDC UserInfo (Bearer; requires <code>scope:openid</code>)
POST /oauth/introspect	RFC 7662 (client-authenticated)
POST /oauth/revoke	RFC 7009 — refresh-family revoke + JWT <code>jti</code> deny-list
GET /oauth/jwks	RFC 7517 JWKS (RSA + EC)
/.well-known/oauth-authorization-server	RFC 8414 discovery
/.well-known/openid-configuration	OIDC discovery
GET /oauth/scopes	scope catalogue with descriptions (<code>ScopeRegistry</code>) — public
GET/POST/PUT/DELETE /oauth/clients	self-service client mgmt (<code>auth-gated</code> , owner-scoped)
GET/DELETE /oauth/authorized-tokens	self-service authorized-apps (list/revoke granted apps)

CSRF. The machine POSTs (`/oauth/token`, `/introspect`, `/revoke`, `/device_authorization`) MUST be in `CsrfTokenLayer exemptPaths` — they are client-authenticated, not cookie-authenticated. The browser consent forms (`/oauth/authorize`, `/oauth/device`) stay CSRF-protected.

5 Tokens

- **Access token = JWT** signed with the platform key (`JWT_ALGO` + keys), so Auth's `JwtAuthLayer` validates it directly. Claims: `iss`; `aud` = the *resource* audience (`OAUTH_TOKEN_AUDIENCE` / `JWT_AUDIENCE`, NOT the client); `azp` = client; `sub`; `scope`; `jti`; and `permissions` as `scope:<name>` — namespaced so an OAuth scope can NEVER satisfy a first-party `hasPermission('admin')`.
- **id_token** (OIDC) — issued when `openid` is granted; carries `nonce`, `aud` = `client_id`, `auth_time`. Refused for a *public* client under symmetric (HS) signing — OIDC needs RS/ES/PS.
- **Refresh token** — opaque, stored hashed, rotating with family reuse-detection.

Because scopes ride as `scope:*` permissions, a resource server checks them with Auth's hierarchical `Guard::hasScope('reports:export')` — an OAuth token can never satisfy a bare RBAC permission.

6 Application services + ports

- `TokenService` — the `/token` endpoint; dispatches every grant, authenticates the client, issues JWT + rotating refresh.
- `AuthorizationService` — the `/authorize` consent flow (stores the request server-side, returns an opaque `authz_id`).
- `DeviceService` — RFC 8628 device flow.
- `IntrospectionService` — RFC 7662 introspection + RFC 7009 revoke (JWT `jti` deny-list, shared with Auth's `JwtAuthLayer`).
- `TokenIssuer` — mints the signed JWT (same keys Auth verifies) + opaque refresh token.
- `ScopeValidator` — requested scopes are registered/grantable.
- `ScopeRegistry` — consent-facing catalogue with descriptions: `scopes()`, `scopesFor()`, `hasScope()`, `tokensCan()`.
- Persistence ports (central): `ClientStore`, `AuthCodeStore`, `RefreshTokenStore`, `ScopeStore`, `DeviceCodeStore`.

7 Self-service management API

The GDA-native port of Passport's `Client` / `AuthorizedAccessToken` / `Scope` controllers. All owner-scoped to the caller's Identity.

7.1 Clients (`/oauth/clients`)

```
GET    /oauth/clients      -> list MY clients (auth)
POST   /oauth/clients    { name, redirect_uris[], scopes[], public? }
                                -> 201 { id, client_secret? } (secret shown ONCE)
PUT     /oauth/clients/{id} { name, redirect_uris[], scopes[] } (mine, else 404)
DELETE /oauth/clients/{id} -> 204 revoke (mine, else 404)
```

A confidential client is issued a secret once on create; a public client gets none and must use PKCE. `owner_id` scopes every read/write to the caller — another owner's client id returns 404, never leaks.

7.2 Authorized apps (`/oauth/authorized-tokens`)

```
GET    /oauth/authorized-tokens -> apps I have authorized (active grants)
DELETE /oauth/authorized-tokens/{id} -> revoke a grant (whole rotation family)
```

7.3 Scopes (`/oauth/scopes`)

Public catalogue with human-readable descriptions, driven by `ScopeStore::describe()` through `ScopeRegistry` — use it to render a consent screen.

8 Usage recipes

8.1 Register a client (CLI or API)

```
# CLI (control plane)
hkm oauth:client:create --name="Acme Dashboard" \
  --redirect="https://acme.example.com/callback" --scope="openid profile"
# --public for a PKCE SPA/mobile client (no secret)
```

```
# or the self-service API (owner = the logged-in user)
POST /oauth/clients { "name": "Acme", "redirect_uris": ["https://.../cb"], "public":
    true }
```

8.2 Authorization code + PKCE (public SPA)

```
// 1. Browser -> /oauth/authorize?response_type=code&client_id=...&redirect_uri=...
//    &scope=openid%20profile&code_challenge=<S256>&code_challenge_method=S256&state
//    =...
//    (user is session-authenticated; consents; server returns ?code=...&state=...)

// 2. SPA exchanges the code (no secret; PKCE verifier proves possession):
POST /oauth/token
  grant_type=authorization_code&code=<code>&redirect_uri=<exact>
  &client_id=<id>&code_verifier=<verifier>
// -> { access_token (JWT), token_type: "Bearer", expires_in, refresh_token, id_token?
    }
```

8.3 Client credentials (server-to-server)

```
POST /oauth/token (HTTP Basic: client_id:client_secret)
  grant_type=client_credentials&scope=reports:read
// -> { access_token, expires_in } (no refresh token; sub = client_id)
```

8.4 Refresh + rotation

```
POST /oauth/token
  grant_type=refresh_token&refresh_token=<opaque>&client_id=<id>[&client_secret=...]
// -> new access token + NEW refresh token (old one revoked). Replaying the old
//    refresh token is REUSE -> the whole family is revoked.
```

8.5 Device code (TV / CLI)

```
POST /oauth/device_authorization client_id=<id>&scope=openid
// -> { device_code, user_code, verification_uri, interval, expires_in }
// user visits /oauth/device, enters user_code, consents; meanwhile the device polls:
POST /oauth/token grant_type=urn:ietf:params:oauth:grant-type:device_code
  &device_code=<...>&client_id=<id>
// -> authorization_pending | slow_down | access_denied | expired_token | tokens
```

8.6 Introspect / revoke (resource server & logout)

```
POST /oauth/introspect token=<access|refresh> (client-authenticated)
// -> { active: true, scope, client_id, sub, exp, ... } | { active: false }

POST /oauth/revoke token=<access|refresh>
// refresh -> revoke the family; access JWT -> deny-list its jti (shared with Auth)
```

8.7 Consuming an OAuth token in a resource server

No OAuth-specific code needed — it is a platform JWT:

```
// Route protected by the 'auth' filter (JwtAuthLayer already verified it):
```

```
if (!$this->guard()->hasScope('reports:export')) { // scope:* -> hierarchical
    return Response::forbidden();
}
```

9 CLI

```
oauth:client:create  --public for PKCE clients; secret shown once
oauth:client:list
oauth:client:revoke
oauth:client:rotate  rotate a confidential client's secret
oauth:prune [--watch=SECONDS]  delete expired codes / refresh / device rows
```

10 Configuration (env)

Key	Meaning
OAuth_ACCESS_TTL	access-token lifetime
OAuth_REFRESH_TTL	refresh-token lifetime
OAuth_CODE_TTL	authorization-code lifetime (default 60s)
OAuth_DEVICE_TTL	device-code lifetime
OAuth_DEVICE_INTERVAL	device poll interval
OAuth_TOKEN_AUDIENCE	resource audience (defaults to JWT_AUDIENCE)

Signing keys come from Auth's JWT_* — use **RS256 + key files** for OIDC (public clients need a verifiable id.token).

11 Rules — do and don't

Do

- Serve /oauth/* on the apex/central host; set TENANCY_BASE_DOMAINS in host mode.
- Rely on JwtAuthLayer to verify access tokens — no OAuth-specific resource-server code.
- Scopes ride in scope AND namespaced scope:* permissions — never bare RBAC names.
- redirect_uri exact match, validated before any error redirect; PKCE mandatory for public clients.
- Refresh rotation with family reuse-detection; auth codes single-use (burned on PKCE/redirect failure).
- OIDC (public clients) requires asymmetric signing (RS/ES/PS) + key files.

Don't

- Put OAuth scopes into bare permissions (collision with first-party authz).
- CSRF-protect the machine token/introspect/revoke endpoints (they are client-authenticated).
- Add a new vendor OAuth package — this server is native on firebase/php-jwt.
- Confuse oauth_clients (applications) with Auth's personal_access_tokens (user API keys).

Reference guide: docs/ai-context/26_OAUTH2.md • First-party auth: Plugins\Auth
(docs/ai-context/25_AUTH.md)